KMS ESP System Report

Cabinet Management System — ESP32 Captive-Portal + Secure UART Relay

Project path: `docs/KMS_ESP_report.pdf`

---

1. Hardware Overview

The KMS system uses two ESP32 microcontroller boards, each serving a distinct role in the cabinet management pipeline:

- **Cabinet ESP:** Creates a Wi-Fi access point (SoftAP) that serves the KMS web UI to any connected user, providing captive portal behavior.
- **Electronics ESP:** Connected to the Cabinet ESP via UART (RX/TX) and receives encrypted action messages to operate relays or other electronics.

User Flow

- A user connects their phone or computer to the Cabinet ESP Wi-Fi SSID (e.g., `ESP-KMS-XXXXX`).
- Opening the browser redirects them to the local KMS website (captive portal). They log in and press action buttons (e.g., *Open Cabinet*).
- The Cabinet ESP encrypts the action and sends it to the Electronics ESP over UART.
- The Electronics ESP verifies and decrypts the message, then triggers connected hardware (e.g., unlocks a cabinet).

Security Overview

- Wi-Fi network is protected with **WPA2** and a strong password.
- Action messages are encrypted using **AES-CTR** and authenticated with **HMAC-SHA256**.
- Each device should have its own unique **32-byte shared key**.

2. Files Included

A) Cabinet ESP — SoftAP + Captive Portal + Encrypt & Send over UART

`cabinet_esp.ino`

This sketch creates a WPA2 SoftAP, runs a DNS server to redirect all DNS queries (captive portal), and serves the KMS web UI from LittleFS via AsyncWebServer. It exposes two API endpoints: `/api/login` (returns a session token) and `/api/action` (requires session token, encrypts the payload with AES-CTR + HMAC-SHA256, and sends it over UART to the Electronics ESP).

```
#include <WiFi.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DNSServer.h>
#include "LittleFS.h"
#include "mbedtls/aes.h"
#include "mbedtls/md.h"

#define AP_SSID_PREFIX  "ESP-KMS-"
#define AP_PASSWORD     "ReplaceWithStrongPassword!"   // WPA2 password
#define DNS_PORT        53
#define HTTP_PORT       80

#define UART_TX_PIN 17
#define UART_RX_PIN 16
#define UART_BAUD   115200

// 32-byte shared key — replace with a secure random key per device
const uint8_t SHARED_KEY[32] = {
  0x12,0x34,0x56,0x78,0x9a,0xbc,0xde,0xf0,
  0x01,0x23,0x45,0x67,0x89,0xab,0xcd,0xef,
  0x10,0x32,0x54,0x76,0x98,0xba,0xdc,0xfe,
  0xaa,0xbb,0xcc,0xdd,0xee,0xff,0x11,0x22
};

DNSServer dnsServer;
AsyncWebServer server(HTTP_PORT);

String currentSessionToken = "";
uint32_t sessionExpiry = 0;

String genToken(size_t len=32) {
  static const char *chars =
    "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  String s; s.reserve(len);
  for (size_t i=0;i<len;i++) s += chars[esp_random() % 62];
  return s;
}

bool aes_ctr_encrypt_and_hmac(
    const uint8_t *key32, const uint8_t *plaintext, size_t plen,
    uint8_t *nonce16, uint8_t *ciphertext, uint8_t *mac32)
{
  for (int i=0;i<4;i++) ((uint32_t*)nonce16)[i] = esp_random();
  mbedtls_aes_context aes; mbedtls_aes_init(&aes);
```

```
  if (mbedtls_aes_setkey_enc(&aes, key32, 128) != 0)
    { mbedtls_aes_free(&aes); return false; }
  unsigned char stream_block[16], nonce_counter[16];
  memcpy(nonce_counter, nonce16, 16);
  unsigned int nc_off = 0;
  mbedtls_aes_crypt_ctr(&aes, plen, &nc_off, nonce_counter,
                        stream_block, plaintext, ciphertext);
  mbedtls_aes_free(&aes);
  // HMAC-SHA256 over (nonce || ciphertext)
  mbedtls_md_context_t ctx;
  const mbedtls_md_info_t *info =
    mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
  mbedtls_md_init(&ctx);
  mbedtls_md_setup(&ctx, info, 1);
  mbedtls_md_hmac_starts(&ctx, key32, 32);
  mbedtls_md_hmac_update(&ctx, nonce16, 16);
  mbedtls_md_hmac_update(&ctx, ciphertext, plen);
  mbedtls_md_hmac_finish(&ctx, mac32);
  mbedtls_md_free(&ctx);
  return true;
}

void uart_send_frame(const uint8_t *payload, size_t payload_len) {
  uint8_t nonce[16];
  uint8_t *ciphertext = (uint8_t*)malloc(payload_len);
  uint8_t mac[32];
  if (!aes_ctr_encrypt_and_hmac(SHARED_KEY, payload,
                                payload_len, nonce, ciphertext, mac))
    { free(ciphertext); return; }
  uint16_t total = 16 + payload_len + 32;
  Serial1.write(0xAA); Serial1.write(0x55);
  Serial1.write((total >> 8) & 0xFF); Serial1.write(total & 0xFF);
  Serial1.write(nonce, 16);
  Serial1.write(ciphertext, payload_len);
  Serial1.write(mac, 32);
  free(ciphertext);
}

bool checkSession(AsyncWebServerRequest *request) {
  if (!request->hasHeader("X-Session-Token")) return false;
  String token = request->header("X-Session-Token")->value();
  return (token == currentSessionToken && millis() <= sessionExpiry);
}

void setup() {
  Serial.begin(115200);
  Serial1.begin(UART_BAUD, SERIAL_8N1, UART_RX_PIN, UART_TX_PIN);
  if (!LittleFS.begin()) Serial.println("LittleFS mount failed");

  String chip = String((uint32_t)ESP.getEfuseMac(), HEX);
  String apSsid = String(AP_SSID_PREFIX) + chip.substring(chip.length()-6);
  WiFi.mode(WIFI_AP);
  WiFi.softAP(apSsid.c_str(), AP_PASSWORD);
  IPAddress apIP = WiFi.softAPIP();
  dnsServer.start(DNS_PORT, "*", apIP);

  server.serveStatic("/", LittleFS, "/").setDefaultFile("index.html");
```

```
  server.on("/api/login", HTTP_POST, [](AsyncWebServerRequest *req){
    if (req->hasParam("plain", true)) {
      String body = req->getParam("plain", true)->value();
      if (body.indexOf("kmsadminpw") != -1) {   // replace with hashed check
        currentSessionToken = genToken(40);
        sessionExpiry = millis() + 30*60*1000;
        req->send(200, "application/json",
          "{"token":"" + currentSessionToken + ""}");
        return;
      }
    }
    req->send(401, "application/json", "{"error":"invalid"}");
  });

  server.on("/api/action", HTTP_POST, [](AsyncWebServerRequest *req){
    if (!checkSession(req))
      { req->send(403, "application/json", "{"error":"auth"}"); return; }
    if (req->hasParam("plain", true)) {
      String body = req->getParam("plain", true)->value();
      uart_send_frame((const uint8_t*)body.c_str(), body.length());
      req->send(200, "application/json", "{"status":"sent"}");
      return;
    }
    req->send(400, "application/json", "{"error":"no_body"}");
  });

  server.onNotFound([](AsyncWebServerRequest *req){
    AsyncWebServerResponse *r =
      req->beginResponse(LittleFS, "/index.html", "text/html");
    r->addHeader("Cache-Control","no-cache, no-store, must-revalidate");
    req->send(r);
  });

  server.begin();
}

void loop() { dnsServer.processNextRequest(); }
```

B) Electronics ESP — UART Receive, Verify HMAC, Decrypt, Act

`electronics_esp.ino`

This sketch listens on Serial1 for framed encrypted messages from the Cabinet ESP. The frame format is: `0xAA 0x55 len_hi len_lo [nonce16][ciphertext][mac32]`. It verifies the HMAC-SHA256 tag, then decrypts with AES-CTR, and prints the action payload (hook electronics control logic at the marked TODO).

```
#include <Arduino.h>
#include "mbedtls/aes.h"
#include "mbedtls/md.h"

#define UART_RX_PIN 16
#define UART_TX_PIN 17
#define UART_BAUD   115200

const uint8_t SHARED_KEY[32] = {
  // MUST match cabinet ESP key exactly
  0x12,0x34,0x56,0x78,0x9a,0xbc,0xde,0xf0,
  0x01,0x23,0x45,0x67,0x89,0xab,0xcd,0xef,
  0x10,0x32,0x54,0x76,0x98,0xba,0xdc,0xfe,
  0xaa,0xbb,0xcc,0xdd,0xee,0xff,0x11,0x22
};

void setup() {
  Serial.begin(115200);
  Serial1.begin(UART_BAUD, SERIAL_8N1, UART_RX_PIN, UART_TX_PIN);
  Serial.println("Electronics ESP ready");
}

bool aes_ctr_decrypt_and_verify(
    const uint8_t *key32, const uint8_t *nonce16,
    const uint8_t *ciphertext, size_t clen,
    const uint8_t *mac32, uint8_t *out_plain)
{
  // Verify HMAC-SHA256
  mbedtls_md_context_t ctx;
  const mbedtls_md_info_t *info =
    mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
  mbedtls_md_init(&ctx);
  mbedtls_md_setup(&ctx, info, 1);
  mbedtls_md_hmac_starts(&ctx, key32, 32);
  mbedtls_md_hmac_update(&ctx, nonce16, 16);
  mbedtls_md_hmac_update(&ctx, ciphertext, clen);
  uint8_t calc_mac[32];
  mbedtls_md_hmac_finish(&ctx, calc_mac);
  mbedtls_md_free(&ctx);
  if (memcmp(calc_mac, mac32, 32) != 0)
    { Serial.println("HMAC mismatch"); return false; }

  // AES-CTR decrypt
  mbedtls_aes_context aes; mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_enc(&aes, key32, 128);
  unsigned char stream_block[16], nonce_counter[16];
  memcpy(nonce_counter, nonce16, 16);
  unsigned int nc_off = 0;
  mbedtls_aes_crypt_ctr(&aes, clen, &nc_off, nonce_counter,
```

```
                        stream_block, ciphertext, out_plain);
  mbedtls_aes_free(&aes);
  return true;
}

void processFrame(const uint8_t *frame, size_t len) {
  if (len < 48) return;
  size_t clen = len - 48;
  const uint8_t *nonce      = frame;
  const uint8_t *ciphertext = frame + 16;
  const uint8_t *mac        = frame + 16 + clen;
  uint8_t *plain = (uint8_t*)malloc(clen + 1);
  if (!aes_ctr_decrypt_and_verify(SHARED_KEY, nonce,
                                  ciphertext, clen, mac, plain))
    { free(plain); return; }
  plain[clen] = 0;
  Serial.print("Received action payload: ");
  Serial.println((char*)plain);
  // TODO: parse JSON and actuate electronics here
  free(plain);
}

void loop() {
  static enum { WAIT_SYNC1, WAIT_SYNC2,
                READ_LEN_HI, READ_LEN_LO, READ_PAYLOAD }
    state = WAIT_SYNC1;
  static uint16_t payloadLen = 0;
  static uint8_t *buf = nullptr;
  static size_t bufPos = 0;

  while (Serial1.available()) {
    uint8_t b = Serial1.read();
    switch (state) {
      case WAIT_SYNC1:  if (b==0xAA) state=WAIT_SYNC2; break;
      case WAIT_SYNC2:  if (b==0x55) state=READ_LEN_HI;
                        else state=WAIT_SYNC1; break;
      case READ_LEN_HI: payloadLen=((uint16_t)b)<<8;
                        state=READ_LEN_LO; break;
      case READ_LEN_LO: payloadLen|=b;
                        if (payloadLen>4096) { state=WAIT_SYNC1; break; }
                        buf=(uint8_t*)malloc(payloadLen);
                        bufPos=0; state=READ_PAYLOAD; break;
      case READ_PAYLOAD: buf[bufPos++]=b;
                        if (bufPos>=payloadLen) {
                          processFrame(buf,payloadLen);
                          free(buf); buf=nullptr;
                          state=WAIT_SYNC1;
                        } break;
    }
  }
}
```

C) Sample Web UI for LittleFS

Two small files — `index.html` and `app.js` — are uploaded to the Cabinet ESP's LittleFS filesystem and served to the connected user's browser.

index.html

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>KMS Portal</title>
</head>
<body>
  <h1>KMS Portal</h1>
  <div id="login">
    <input id="pw" type="password" placeholder="admin password" />
    <button id="btnLogin">Login</button>
  </div>
  <div id="actions" style="display:none;">
    <button id="btnOpen">Open Cabinet</button>
    <button id="btnClose">Close Cabinet</button>
  </div>
  <pre id="out"></pre>
  <script src="/app.js"></script>
</body>
</html>
```

app.js

```
async function postJSON(url, obj, token) {
  const headers = { 'Content-Type': 'application/json' };
  if (token) headers['X-Session-Token'] = token;
  const res = await fetch(url, {
    method: 'POST', body: JSON.stringify(obj), headers
  });
  return res.json().catch(() => ({}));
}

let token = null;

document.getElementById('btnLogin').addEventListener('click', async () => {
  const pw = document.getElementById('pw').value;
  const r  = await postJSON('/api/login', { password: pw });
  if (r.token) {
    token = r.token;
    document.getElementById('login').style.display   = 'none';
    document.getElementById('actions').style.display = 'block';
    log('Logged in');
  } else { log('Login failed'); }
});

document.getElementById('btnOpen').addEventListener('click', async () => {
  if (!token) return log('Not logged in');
  const r = await postJSON('/api/action', { action: 'open' }, token);
  log('Action sent: ' + JSON.stringify(r));
});
```

```
document.getElementById('btnClose').addEventListener('click', async () => {
  if (!token) return log('Not logged in');
  const r = await postJSON('/api/action', { action: 'close' }, token);
  log('Action sent: ' + JSON.stringify(r));
});

function log(s) { document.getElementById('out').textContent += s + "\n"; }
```

3. How to Deploy

Prerequisites

- Two ESP32 boards + USB cables + a computer with Arduino IDE or PlatformIO
- ESP32 board support installed in Arduino IDE or PlatformIO
- LittleFS uploader plugin (Arduino) or PlatformIO filesystem upload
- Jumper wires for UART (TX→RX) and a common ground between boards

Edit Configuration in Code

- Replace `AP_PASSWORD` with a strong WPA2 password in `cabinet_esp.ino`.
- Replace `SHARED_KEY` (32 bytes) in **both** sketches with a secure random key — must match on both devices.
- Replace `expectedPass` in the cabinet sketch, or implement a hashed password check.
- Adjust UART pin definitions if needed.

Upload Web Files to LittleFS (Cabinet ESP)

Place `index.html` and `app.js` into the LittleFS root directory and upload using the LittleFS tool in your IDE.

Flash Sketches

- Select the correct ESP32 board and COM port in Arduino IDE.
- Upload `cabinet_esp.ino` to the Cabinet board.
- Upload `electronics_esp.ino` to the Electronics board.

Wiring

- Connect Cabinet Serial1 TX pin → Electronics RX pin.
- Optionally connect Electronics TX → Cabinet RX for bi-directional comms.
- Connect ground between both boards.

Test Flow

- Connect your phone to the Wi-Fi SSID `ESP-KMS-xxxx`.
- Open a browser — the captive portal should redirect to `index.html`.
- Log in and press action buttons.
- Monitor Serial on the Electronics ESP to see decrypted payloads.

4. Troubleshooting

Symptom	Resolution

Captive portal not appearing	Open http://192.168.4.1 directly in the browser
HMAC mismatch	Verify SHARED_KEY bytes match exactly on both boards
UART frames not arriving	Check TX/RX wiring and confirm matching baud rate (115200)
LittleFS not serving files	Verify LittleFS upload succeeded and index.html is present

5. Security Checklist

All items below are **must-do** before deploying in any production environment:

✓ **Unique keys per device**

Provision each Cabinet ESP / Electronics ESP pair with a different SHARED_KEY generated with a cryptographically secure random source.

✓ **Strong WPA2 password**

Replace the placeholder AP_PASSWORD with a password of at least 16 characters containing mixed case, digits, and symbols.

✓ **Session tokens & short expiry**

Tokens are already randomised (40-char alphanumeric). The default 30-minute expiry should be shortened for higher-security deployments.

✓ **Signed firmware updates**

If OTA updates are enabled, verify firmware signatures to prevent malicious replacement of sketches.

✓ **TLS gateway (optional)**

If browser-trusted HTTPS is required, place a TLS-terminating gateway in front of the SoftAP or use ESP32's SSL libraries.

6. Next Steps & Notes

- For production deployments, move to a more robust authentication system, secure key provisioning, and signed OTA firmware updates.
- Consider **ESP-NOW** or **MQTT-over-TLS** if scalable networking is needed instead of direct UART.

End of KMS ESP System Report.